

Resumen Teorico – Practico 8

Dobles de prueba, generacion automatica y ejecucion
simbolica

Testing de Software 2024

Índice

Introduccion	3
1. Dobles de prueba (Mocking)	3
1.1. Por que se necesitan	3
1.2. Entrada y salida indirectas	4
1.3. Estrategias de integracion	4
1.4. Taxonomia de dobles	4
1.5. Cuando usar cada uno	5
2. Generacion automatica de tests	5
2.1. Idea general	5
2.2. Randoop: generacion aleatoria guiada por feedback	6
2.3. EvoSuite: busqueda evolutiva	7
2.4. Limitaciones comunes	7
3. Ejecucion simbolica (Symbolic Execution)	8
3.1. Idea	8
3.2. Ejemplo	8
3.3. Ejecucion simbolica <i>dinamica</i> (concolic execution)	9
3.4. Constraint solvers	9
3.5. Limitaciones de SE	10
4. Relacion entre las tres tecnicas	10
5. Comparacion con criterios anteriores	10

6. Glosario	11
Conclusion	11

Introduccion

Este resumen sintetiza el material teorico asociado al TP8:

- *Notas 14: Dobles de pruebas (Mocking)* (V. Bengolea).
- *Notas 15: Generacion automatica de tests — EvoSuite y algoritmos evolutivos* (V. Bengolea).
- *Notas 16: Ejecucion simbolica dinamica* (V. Bengolea).

Tres ideas centrales que se cruzan.

- **Dobles de prueba:** como aislar la clase bajo test de sus colaboradores cuando estos son lentos, indeterministas o todavia no estan implementados.
- **Generacion automatica de tests:** dos enfoques representativos — *Randoop* (generacion aleatoria guiada por feedback) y *EvoSuite* (busqueda evolutiva guiada por cobertura).
- **Ejecucion simbolica:** una tecnica que recorre el programa con *valores simbolicos* en lugar de concretos, recolecta *condiciones de camino* y las resuelve con un *constraint solver* para fabricar inputs que recorran caminos especificos.

Las tres ideas se complementan: los mocks resuelven dependencias estaticas, la generacion automatica produce volumen de tests, y la ejecucion simbolica garantiza precision en la exploracion de caminos dificiles.

1. Dobles de prueba (Mocking)

1.1. Por que se necesitan

Cuando una unidad bajo prueba (SUT, *System Under Test*) depende de otros componentes (*Depended-On Components*, DOCs), aparecen dos problemas:

- Los DOCs pueden ser *lentos* (red, base de datos), *indeterministas* (reloj, IO), o *costosos* (servicios pagos).
- Los DOCs pueden no estar *implementados todavia*, especialmente en desarrollo agil.

Definicion

Un **doble de prueba** (*test double*) es un esqueleto o implementacion especial de un modulo que se necesita para testear un componente que depende de el. Reemplaza al DOC para permitir que el SUT pueda probarse de manera aislada.

1.2. Entrada y salida indirectas

El SUT interactúa con sus DOCs de dos maneras:

- **Entrada indirecta:** el SUT *lee* un valor del DOC (e.g. `doc.meth(...)` devuelve algo que el SUT usa en su lógica).
- **Salida indirecta:** el SUT *modifica* el estado del DOC (e.g. `db.save(o)`).

Idea clave

Un doble reemplaza al DOC para *controlar las entradas indirectas* (forzar que el SUT reciba el valor que necesitamos) y/o para *monitorear las salidas indirectas* (verificar que el SUT invoca al DOC correctamente).

1.3. Estrategias de integración

Adentro hacia afuera (inside-out). Se codifica y prueba primero el núcleo y se va hacia las capas externas. Minimiza la necesidad de dobles pero minimiza también el feedback temprano. Si los requisitos cambian, hay que re trabajar tests.

Afuera hacia adentro (outside-in). Se codifica y prueba desde las capas externas hacia el núcleo, usando dobles que se van reemplazando a medida que se completa la implementación. Maximiza dobles y feedback, pero minimiza el trabajo adicional ante cambios de requisitos. Es el patrón natural en desarrollo ágil.

1.4. Taxonomía de dobles

Hay cinco tipos principales (según Gerard Meszaros, *xUnit Test Patterns*):

1. **Dummy:** solo satisface dependencias formales. Se pasa como parámetro pero no hace nada; en general lanza excepción si se lo usa. Útil cuando un constructor requiere un parámetro que el escenario no ejerce.
2. **Stub:** provee respuestas *prefabricadas* para las invocaciones que el SUT le hace. Controla las entradas indirectas.
3. **Spy:** como un stub, pero además *registra* las invocaciones recibidas para inspeccionarlas después. También permite monitorear salidas indirectas.
4. **Fake:** implementación alternativa funcional pero simplificada (e.g. una base de datos en memoria en vez de la real). Suficiente para correr tests pero no para producción.
5. **Mock:** doble pre-programado con expectativas exactas sobre que métodos se le van a llamar, con que argumentos y en que orden. Si las expectativas no se cumplen, el test

falla. Frameworks como EasyMock y Mockito lo automatizan.

Ejemplo

Con EasyMock:

```
LoginService mock = createMock(LoginService.class);
expect(mock.login("ip", "user", "hash")).andReturn(false).
    times(3);
replay(mock);
// ... ejercitar el SUT que usa el mock ...
verify(mock); // falla si las invocaciones no se cumplieron
```

1.5. Cuando usar cada uno

- **Dummy**: cuando el parametro no influye en el escenario.
- **Stub**: cuando solo se necesita una respuesta canned.
- **Spy**: cuando ademas se quiere verificar que el SUT llamo correctamente.
- **Fake**: cuando la integracion real es demasiado pesada pero hay logica significativa a ejercitar.
- **Mock**: cuando la *interaccion* entre SUT y DOC es la propiedad central que se quiere verificar (e.g. “el SUT debe llamar a **save** exactamente una vez”).

Idea clave

Regla practica: usar el doble *mas simple* que sirva. Empezar con **stub**; subir a **mock** solo cuando hace falta verificar interacciones especificas.

2. Generacion automatica de tests

2.1. Idea general

Crear tests a mano es *costoso*, *tedioso* e *incompleto*. La generacion automatica busca *producir tests* a partir del codigo fuente o de su firma, sin intervencion humana en cada caso. Hay varias estrategias; el TP8 cubre dos representativas:

- **Randoop**: generacion aleatoria guiada por feedback.
- **EvoSuite**: busqueda evolutiva guiada por cobertura.

2.2. Randoop: generacion aleatoria guiada por feedback

Algoritmo basico (simplificado).

1. Mantener un *pool* de secuencias previas que terminan en objetos validos.
2. Mientras quede tiempo:
 - a) Elegir un metodo $m(T_1, \dots, T_k) \rightarrow T_{\text{ret}}$ al azar de la API.
 - b) Para cada parametro de tipo T_i , elegir aleatoriamente del pool una secuencia S_i que construya un valor de ese tipo.
 - c) Construir la nueva secuencia $S_{\text{new}} = S_1; S_2; \dots; S_k; T_{\text{ret}}$ $v_{\text{new}} = m(v_1, \dots, v_k);$.
 - d) Si la secuencia ya existia sintacticamente, descartar.
 - e) Ejecutar y *clasificar* la secuencia:
 - Crash inesperado o violacion de contrato basico $\rightarrow$ **ErrorTest** (bug a investigar).
 - Ejecucion exitosa $\rightarrow$ **RegressionTest** (suite de regresion) y se agrega al pool.
 - Excepcion declarada o invalidacion $\rightarrow$ descartar.

Salida.

- **ErrorTest**: tests que rompen contratos basicos (e.g. `equals/hashCode`, `toString` con NPE).
- **RegressionTest**: tests que documentan el comportamiento actual.

Aserciones de regresion. Randoop usa metodos *observadores* (sin side-effects) para capturar el estado actual del objeto y generar asserts del estilo `assertTrue(b2 == false)`. Estos asserts no garantizan correctitud, pero detectan *regresiones* ante cambios futuros.

Mecanismos para mejorar.

- `-junit-before-each/-junit-after-each`: inyectar setup/teardown (e.g. recrear `System.in`, archivos temporales).
- `-system-props`: fijar locale, timezone para reproducibilidad.
- `-methodlist/-omitmethods`: concentrar la exploracion.
- `-timeout`: cortar bloqueos en lecturas de `stdin`.

- `@randoop.CheckRep`: indicar que un metodo es un invariante; Randoop lo evalua en cada secuencia.

2.3. EvoSuite: busqueda evolutiva

Idea. En lugar de generar al azar, EvoSuite mantiene una *poblacion* de suites de test y la evoluciona usando un *algoritmo genetico*:

- **Funcion de fitness**: usualmente cobertura (lineas, ramas, *weak mutation*). Cuanto mas alta, mejor.
- **Cruce y mutacion**: combina suites existentes y aplica pequenas mutaciones (agregar una llamada, cambiar un valor, eliminar una linea).
- **Seleccion**: las suites con mejor fitness sobreviven a la siguiente generacion.

Salida.

- Suite “feliz” (e.g. `Server_ESTest.java`) con tests que pasan.
- Suite “failed” (e.g. `Server_Failed_ESTest.java`) con escenarios que rompen el estado interno.

Diferencia clave con Randoop.

	Randoop	EvoSuite
Tecnica	aleatoria + feedback	busqueda evolutiva
Objetivo	inputs validos / errores	maximizar cobertura
Tiempo de generacion	muy rapido	mas lento (search budget)
Calidad de la suite	superficial	mas profunda
Costo de integracion	bajo (Java moderno)	alto (Java 8/11)

Idea clave

Cuando elegir cada uno. Randoop es ideal como primera pasada: rapido, encuentra crashes evidentes y NPEs. EvoSuite vale la pena cuando se busca cobertura alta de ramas; es mas caro de configurar pero produce suites mas exigentes.

2.4. Limitaciones comunes

- **Oraculo limitado.** Sin propiedades explicitas, las herramientas solo detectan bugs “visibles”: crashes, violaciones de contratos del lenguaje.
- **Tests poco legibles.** Las secuencias generadas son mecanicas y dificiles de interpretar.

- **Dependencias externas.** Cualquier clase que dependa de `System.in`, `filesystem`, red o reloj suele degradar drásticamente la calidad de la generación.

3. Ejecución simbólica (Symbolic Execution)

3.1. Idea

Definición

Ejecución simbólica (SE). Es una técnica que ejecuta el programa con *valores simbólicos* en lugar de concretos. En cada decisión se bifurca: una rama asume que la condición vale `true`, otra que vale `false`. En cada paso se acumulan dos piezas de estado:

- **Estado simbólico:** el valor (fórmula) de cada variable.
- **Condición de camino** (*path condition*): la conjunción de todas las guardas que se asumieron hasta llegar a ese punto.

Cada hoja del árbol simbólico corresponde a un camino del programa. Para fabricar un input concreto que recorra ese camino, se le pasa la *path condition* a un **constraint solver** (e.g. Z3, CVC5).

3.2. Ejemplo

```
public int testMe(int a, int b) {  
    if (b < 0) {  
        return -a;  
    } else {  
        int c = a - b;  
        if (b + c == 400) {  
            throw new IllegalArgumentException();  
        } else {  
            return c;  
        }  
    }  
}
```

Los tres caminos producen tres *path conditions*:

1. $b_0 < 0$ (camino `return -a`).
2. $b_0 \geq 0 \wedge b_0 + (a_0 - b_0) \neq 400$, simplificada $b_0 \geq 0 \wedge a_0 \neq 400$ (camino `return c`).
3. $b_0 \geq 0 \wedge a_0 = 400$ (camino `throw`).

Pasados al constraint solver, dan ejemplos concretos:

- $b_0 = -1$, $a_0 = 0$ (cualquiera).
- $b_0 = 0$, $a_0 = 0$.
- $b_0 = 0$, $a_0 = 400$.

Idea clave

La ejecucion simbolica es *precisa*: cada test generado *garantiza* recorrer un camino especifico. Es lo opuesto a la generacion aleatoria, que puede ejecutar muchas veces el mismo camino y nunca tocar otros.

3.3. Ejecucion simbolica *dinamica* (concolic execution)

La SE pura tiene problemas en presencia de codigo que no se puede analizar simbolicamente (llamadas a librerias nativas, IO, codigo binario). La variante *concolic* (concrete + symbolic) combina ambas:

1. Se elige un input concreto inicial.
2. Se ejecuta el programa concretamente, pero *en paralelo* se mantiene la ejecucion simbolica.
3. Cuando aparece codigo no analizable, se usa el valor concreto.
4. Al terminar, se invierte una guarda del camino y se le pide al solver un input que recorra el camino alternativo.
5. Se repite, ganando un camino nuevo en cada iteracion.

Herramientas tipicas: *KLEE*, *Symbolic PathFinder*, *angr*, *DART*, *CUTE*.

3.4. Constraint solvers

Un **constraint solver** recibe una formula logica (tipicamente en logica de primer orden sobre enteros, bitvectors, arrays, etc.) y responde:

- **SAT**: la formula es satisfacible y devuelve un modelo concreto.
- **UNSAT**: no hay valores que la satisfagan; el camino es *infactible*.
- **UNKNOWN**: el solver no pudo decidir en el tiempo dado.

Los *SMT solvers* modernos (*Satisfiability Modulo Theories*) como Z3 manejan combinaciones de teorias (aritmetica, bitvectors, strings, arrays).

3.5. Limitaciones de SE

- **Path explosion:** el numero de caminos crece exponencialmente con el numero de ramas. Bucles y recursion son particularmente problematicos.
- **Constraints no decidibles:** el solver puede fallar con aritmetica no lineal, strings complejos o llamadas externas.
- **Modelado de heap:** estructuras dinamicas (listas enlazadas, arboles) son dificiles de simbolizar de manera completa.
- **Codigo nativo:** las llamadas a JNI o librerias C escapan al analisis.

4. Relacion entre las tres tecnicas

Tecnica	Que automatiza	Costo
Mocks	aislar el SUT	escribir el doble
Randoop	generar tests aleatorios	casi nulo
EvoSuite	maximizar cobertura	search budget
SE / concolic	explorar caminos especificos	solver + analisis

Idea clave

Las tecnicas no compiten: se combinan.

- Mocks se usan *dentro* de cualquier tipo de test (unit, PBT, generado).
- Randoop y EvoSuite suelen *integrar* dobles automaticamente cuando detectan dependencias.
- La ejecucion simbolica puede usarse para *guiar* la generacion (e.g. ayudando a EvoSuite a encontrar caminos profundos).

5. Comparacion con criterios anteriores

- **Particion del input** (TP2) y **PBT** (TP7) producen tests *escritos a mano* pero *abstractos*; las tecnicas del TP8 los generan automaticamente, con menos garantia semantica.
- **Cobertura estructural** (TP3-TP4) provee la metrica que la busqueda evolutiva de EvoSuite optimiza.
- **Cobertura logica** (TP5) y **mutacion** (TP6) son maneras de *evaluar* una suite generada automaticamente.

- **Fuzzing** (TP6) es el primo pobre de la ejecucion simbolica: tambien busca inputs, pero al azar.

6. Glosario

- **SUT** (*System Under Test*): la pieza de software que se quiere testear.
- **DOC** (*Depended-On Component*): componente del que depende el SUT.
- **Dummy/Stub/Spy/Fake/Mock**: tipos de dobles ordenados de menor a mayor funcionalidad.
- **Path condition**: conjuncion de las guardas asumidas en un camino simbolico.
- **Constraint solver**: herramienta que decide la satisfacibilidad de formulas logicas y produce modelos.
- **SAT/UNSAT/UNKNOWN**: posibles respuestas de un constraint solver.
- **Path explosion**: explosion combinatoria del numero de caminos en SE.
- **Search budget**: presupuesto de tiempo o de evaluaciones de fitness en una busqueda evolutiva.
- **ErrorTest / RegressionTest**: dos clases de tests que produce Randoop.

Conclusion

Los dobles de prueba resuelven el problema de *como* testear unidades acopladas. La generacion automatica de tests (Randoop, EvoSuite) resuelve el problema de *escribir* tests cuando hay mucho codigo. La ejecucion simbolica resuelve el problema de *recorrer* caminos especificos del programa con precision. Ninguna de las tres reemplaza al criterio humano: el oraculo — saber que es “correcto” — sigue siendo responsabilidad del desarrollador. Pero las tres juntas permiten escalar el testing mucho mas alla de lo que un humano podria escribir caso por caso, sin perder el control sobre *que* se esta probando.